Operating Systems and System Programming

Fall 2026

Chapter IV: File Systems

IV.1. Overview

A file system is a method and data structure that an operating system uses to control how data is stored and retrieved.

Without a file system, data placed in a storage medium would be one large block with no way to tell where one piece of data ends and the next begins.

This chapter will cover core aspect of file systems.

Outline

- Generalities on Files
- File systems
- File Systems vs Directory Structure
- Conclusion

Outline

- **Generalities on Files**
- File systems
- File Systems vs Directory Structure
- Conclusion

IV.2. Generalities on Files

IV.2.1. Definition

A file is a **named collection of related information recorded on secondary storage**.

It can represent programs, documents, images, videos, or system data.

IV.2.2. File attributes

Important information of files are:

- **Name** : Human-readable identifier.
- **Type** : Indicates the kind of data (e.g., .txt, .jpg, .exe).
- **Location** : Pointer to the storage location.
- **Size** : Current file size.
- **Protection** : Access control information.
- **Time/Date** : Creation, modification, and access timestamps.

IV.2.3. File Operations

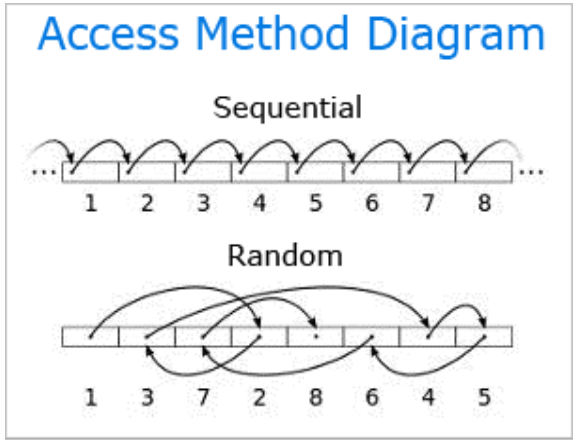
Actions that can be performed on files are:

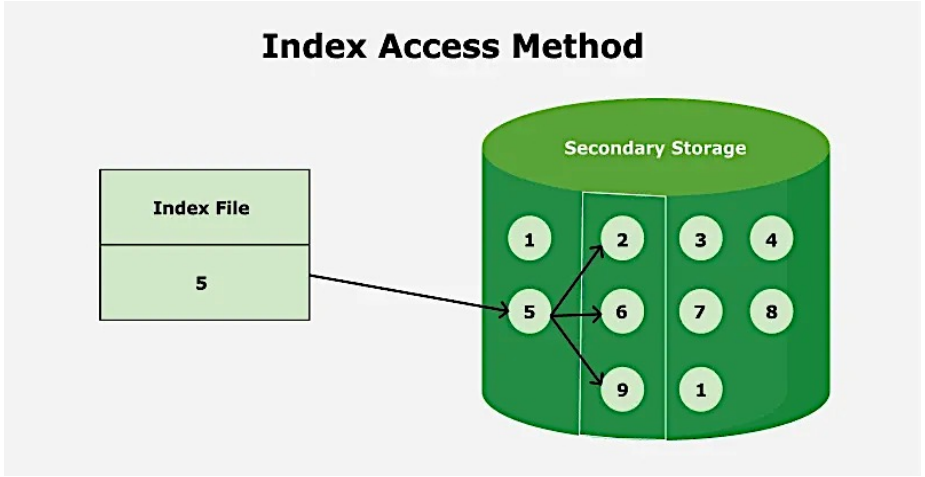
- **Creation** : Allocating space and metadata.
- **Writing** : Adding data to a file.
- **Reading** : Retrieving data from a file.
- **Deleting** : Removing file and free space.
- **Opening/Closing** : Preparing file for access and releasing resources.
- **Seeking** : Moving the file pointer to a specific location within the file.

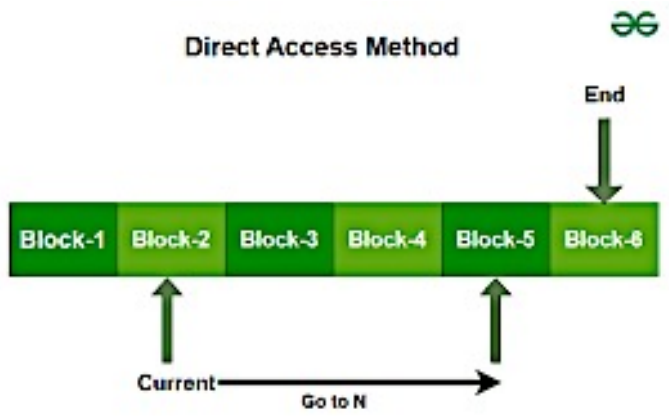
IV.2.4. File Access Methods.

Files can be access through:

- **Sequential access** : Data is accessed in a linear order (*Common in text files*).
- **Direct access** : Allows access to any block of data directly (*Used in databases*).
- **Indexed access** : Uses an index to locate blocks of data (*Efficient for large files with frequent random access*).





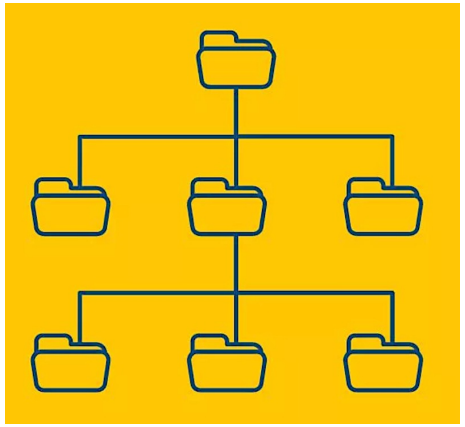


Source: https://www.geeksforgeeks.org/operating-systems/file-access-methods-in-operating-system/

IV.2.5. Directory Structure

A directory is a special type of file used by a file system **to organize and manage other files and directories**. It helps group related files together and makes it easier to locate and manage them.

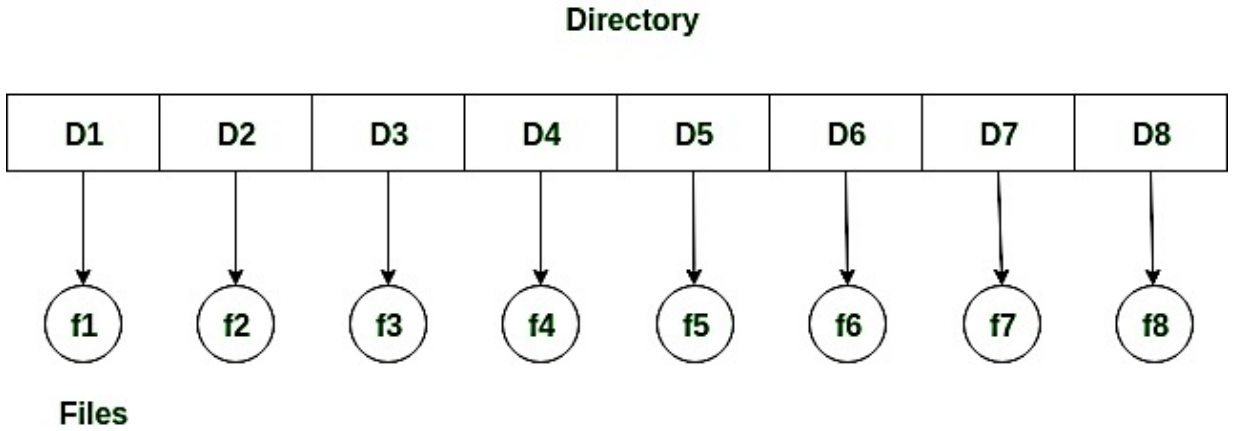
- **Purpose:**
  - Organize files into a hierarchical structure.
  - Enable grouping, searching, and access control.
- **Directory operations**
  - **Create** : Making a new directory to store files
  - **Delete** : Removing a directory (usually empty)
  - **List files** : Showing content of a directory
  - **Rename** : Changing the name of a directory
  - **Traverse** : Moving through directory to access files



A. Logical Structure (from user perspective):

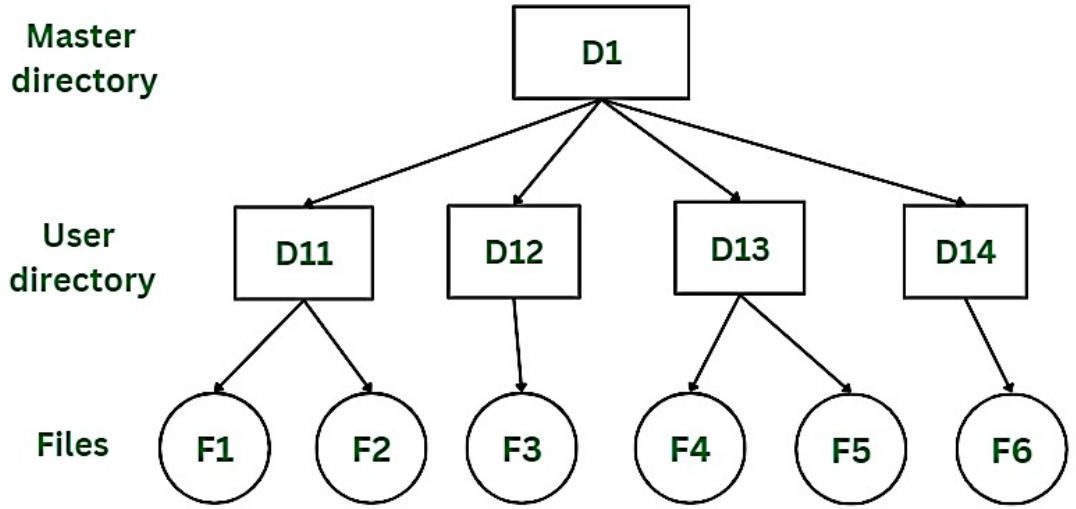
- **Single-Level** : All files in one directory (simplest)
- **Two-Level** : Separate directories for each user (Multi-user systems)
- **Tree-Structured** : Hierarchical directories
- **Acyclic Graph** : Allows shared subdirectories
- **General Graph** : Allows cycles (requires garbage collection)

Levels	Advantages	Disadvantages
Single-Level	• Logical organization: Arrange files hierarchically for easy navigation. • Efficiency: Faster file searching and access. • Security: Restrict access at directory level to protect data. • Backup & Recovery: Simplifies locating and restoring files. • Scalability: Easily supports growth with new files and directories	• There may chance of name collision because two files can have the same name. • Searching will become time taking if the directory is large. • This can not group the same type of files together.



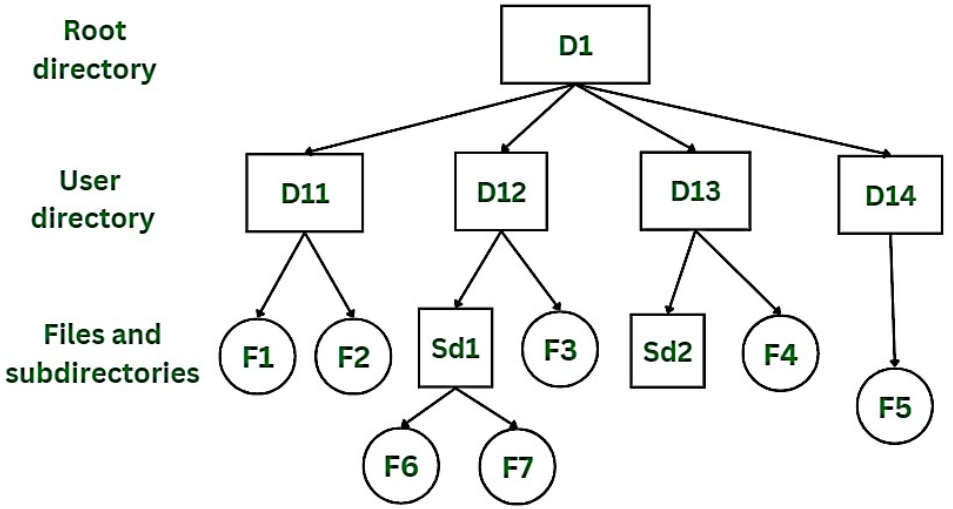
Source: https://www.geeksforgeeks.org/operating-systems/structures-of-directory-in-operating-system/

Levels	Description	Advantages	Disadvantages
Two-Level	Each user has a separate User File Directory (UFD) containing only their files. A Master File Directory (MFD) stores entries for all users and points to their respective UFDs, preventing filename conflicts between users.	• Can have more than two files with same name, and would be very helpful if there are multiple users. • A security would be there which would prevent user to access other user's files. • Searching of the files becomes very easy in this directory structure. • Users can create their own files	• User cannot share the file with the other users. • No ability to create subdirectories. • Scalability is not possible because one user cannot group the same types of files together.



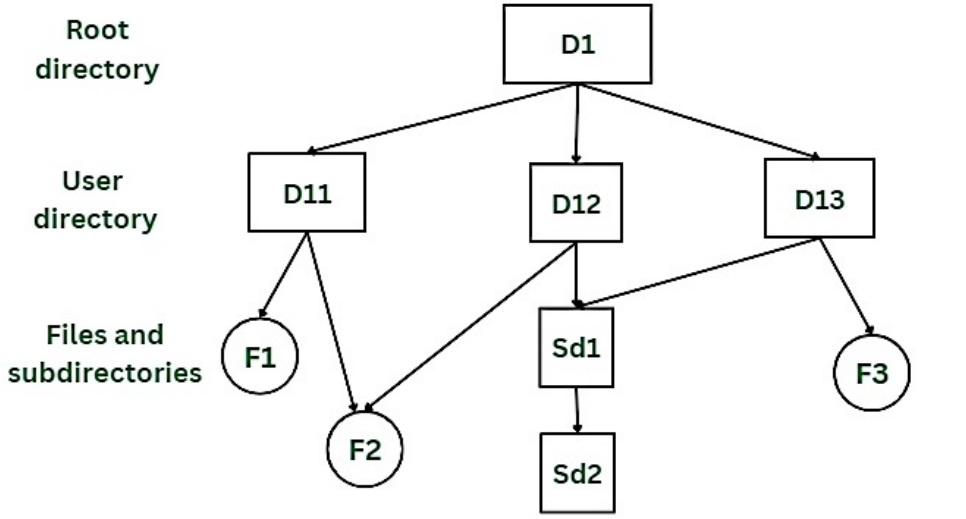
Source: https://www.geeksforgeeks.org/operating-systems/structures-of-directory-in-operating-system/

Levels	Description	Advantages	Disadvantages
Tree-Structure	The most common in personal computers. Each user can create files and subdirectories within their own directory but cannot access or modify the root or other users' directories.	• This directory structure allows subdirectories inside a directory. • The searching is easier. • File sorting of important and unimportant becomes easier. • This directory is more scalable than the other two directory structures explained.	• Prevents the file sharing among users. • With the number of subdirectories increase the searching may become complicated. • Users cannot modify the root directory data.



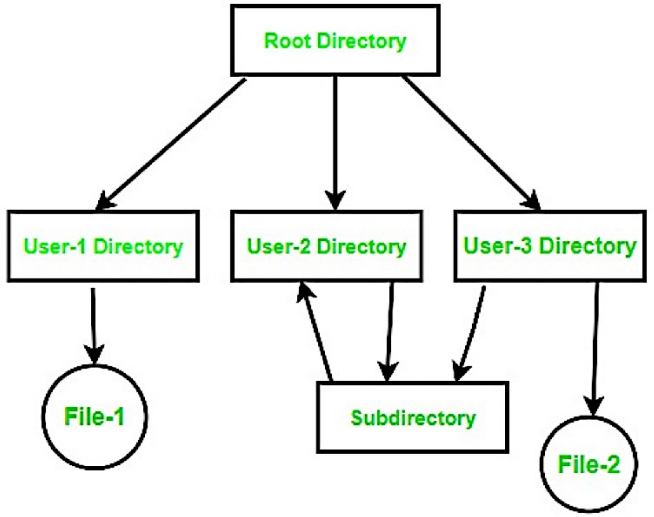
Source: https://www.geeksforgeeks.org/operating-systems/structures-of-directory-in-operating-system/

Levels	Description	Advantages	Disadvantages
Acyclic Graph	• Allows a file or subdirectory to be shared across multiple directories using links. • Changes made by one user are visible to all users sharing that file.	• Sharing of files and directories is allowed between multiple users. • Searching becomes too easy. • Flexibility is increased as file sharing and editing access is there for multiple users.	• Because of the complex structure it has, it is difficult to implement this directory structure. • The user must be very cautious to edit or even deletion of file as the file is accessed by multiple users. • If we need to delete the file, then we need to delete all the references of the file in order to delete it permanently.



Source: https://www.geeksforgeeks.org/operating-systems/structures-of-directory-in-operating-system/

Levels	Description	Advantages	Disadvantages
General Graph	Unlike the acyclic-graph directory, which avoids loops, the general-graph directory can have cycles, meaning a directory can contain paths that loop back to the starting point.	• More flexible than other directory structures. • Allows cycles, meaning directories can loop back to each other.	• More expensive to implement compared to other solutions. • Requires garbage collection to manage and clean up unused files and directories.



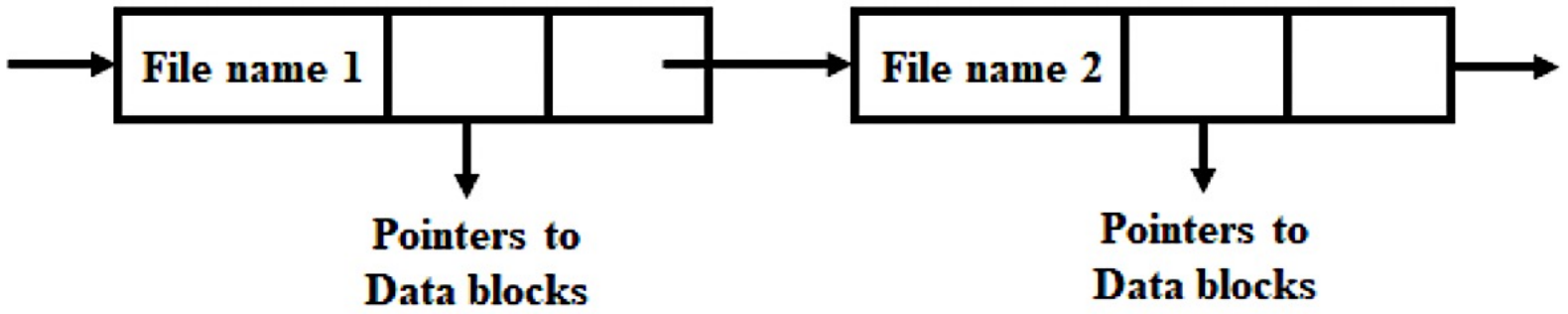
Source: https://www.geeksforgeeks.org/operating-systems/structures-of-directory-in-operating-system/

B. Implementation Structure (How it is stored)

❑**Linear list:**

- Stores directory entries sequentially (simplest of all directory implementation methods but it takes a lot of time to execute).
- It simply maintains a sequential list of file names pointing to their corresponding data blocks.
- Simple but slow for large directories ($O(n)$ search time).

- Requires a filename to be searched before creating a file with that name. Similarly, a delete operation also requires a linear search for the directory and then deallocating the space occupied by the directory. Finally, the corresponding entry is removed from the list.



In a linear list directory, all file entries are stored in **one single list** such as:

Directory:[file1.txt | file2.txt | file3.txt | ... | fileN.txt]

When one want to **find, add, or delete** a file, the system must **scan through the list from the beginning** until it finds what it's looking for.

Example:

Say you want to open fileN.txt. The system does not know where it is, so it checks:

1. Check file1.txt → not it
2. Check file2.txt → not it
3. Check file3.txt → not it...
4. Check fileN.txt → found!

- **Best case**: the file is first → **1 comparison** → O(1)
- **Worst case**: the file is last (or not present) → **N comparisons** → **O(n)**
- **Average case**: on average you check N/2 entries → still **O(n)**
- So the **time to find a file grows linearly** with the number of files n.

Note:

The Basic Idea behind the Big-O notation

It is a way to describe **how the running time (or memory) of an algorithm grows as the input size grows**. It does not tell you the exact number of seconds. Instead, it tells you the **growth pattern**, how the work scales when n (the input size) gets bigger.

Understand it as: *"If I double the input, how much slower does it get?"*

❑Hash table:

It answers:

"What if I could compute exactly where "fileN.txt" *should be, and go straight there?"*

It has two parts:

- **An array** (called "buckets" or "slots")
- **A hash function**: a formula that converts a key into an array index.
- And it is used inside **OS kernel** (in RAM, managed by the kernel), a **user application** (in RAM, managed by the process), a **database**, or a **web browser.**

Example:

- We have an array of 10 slots (indices 0–9):

Index: 0 1 2 3 4 5 6 7 8 9
[| | | | | | | | |]

- A simple hash function for names could be: **sum the letters' positions, then mod by 10**.

"Alice" → A(1)+l(12)+i(9)+c(3)+e(5) = 30 → 30 % 10 = 0 → slot 0

"Bob" → B(2)+o(15)+b(2) = 19 → 19 % 10 = 9 → slot 9

"Dave" → D(4)+a(1)+v(22)+e(5) = 32 → 32 % 10 = 2 → slot 2

- Now the table looks like:

Index: 0 1 2 3 4 5 6 7 8 9
[Alice | |Dave | | | | | | |Bob]

Looking up "Dave”:

Instead of scanning as in lists, one can compute his slot:

"Dave" → hash → 32 % 10 = 2 → go straight to slot 2 → found!

Advantage: One step, That is O(1).

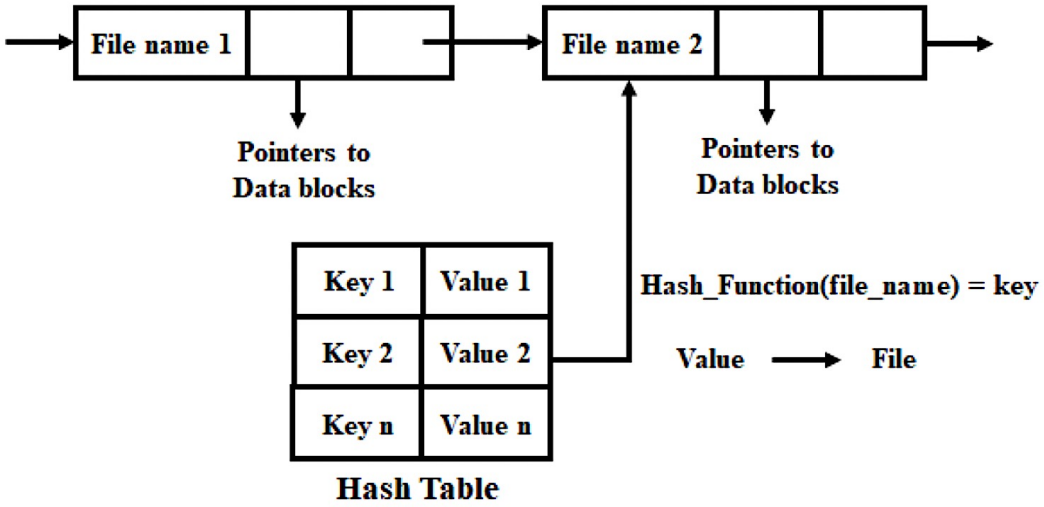
Disadvantage:

- Collisions: two keys hash to have the same slot.
  - "Alice" → slot 0”; Alicia" → also slot 0 ← COLLISION!
  - Can be addressed through: chaining (Each slot holds a **small list**. Both items go in the slot 0's list), or Open addressing (if a slot is taken, **probe** for the next free slot)

With a **good hash function** and **low load factor (number of items / number of slots** (not too full), collisions stay rare, so average performance stays **O(1)**.

- Low load factor → few collisions → fast (close to O(1))
- High load factor → many collisions → slow (approaches O(n))

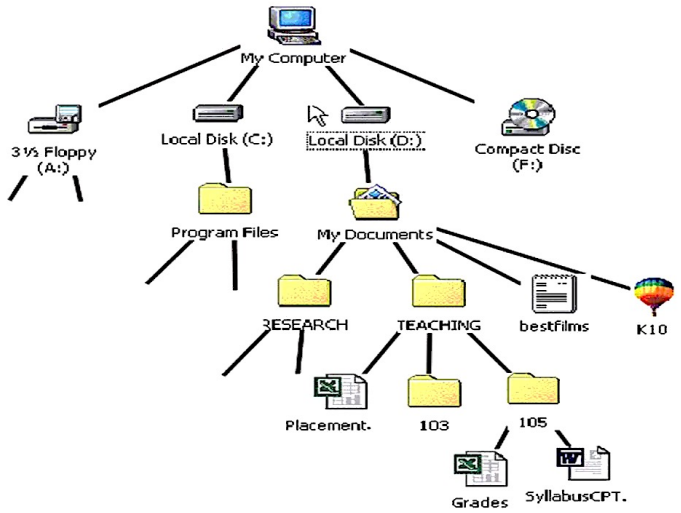
- When the load factor gets too high, hash tables **resize** (grow the array and rehash everything) to keep performance fast.



❑Tree-Based (B-Tree, B+ Tree):

A **tree-based** directory structure organizes files **hierarchically**, folders inside folders inside folders. This is what you actually use every day (Windows Explorer, macOS Finder, Linux filesystem).

Search in a tree can be **O(log n)** (balanced tree) or **O(depth)** (directory tree), much better than the linear list's **O(n)**.



Note:

Pathname resolution

When a user requests a file (e.g., /home/user/file.txt), the OS:

- Starts at the root directory (/).
- Traverses each component (home → user).
- Locates the final file (file.txt) by checking directory entries.

Example:

- Linear list: up to 1,000,000 checks → O(n)
- Balanced tree: about 20 checks → O(log n)

Comparison: Linear vs Hash vs Tree

Structure	Search	Insert	Delete	Ordered?	Scales well?
Linear list	O(n)	O(n)	O(n)	Yes (insertion order)	❌ No
Hash table	O(1) avg	O(1) avg	O(1) avg	❌ No	✅ Yes
Balanced tree	O(log n)	O(log n)	O(log n)	✅ Yes (sorted)	✅ Yes

Outline

- Generalities on files
- **File systems**
- File Systems vs Directory Structure
- Conclusion

IV.3. Files Systems

A file system is a fundamental component of an operating system that **manages how data is stored, organized, and retrieved on storage devices** such as hard disks, SSDs, and USB drives.

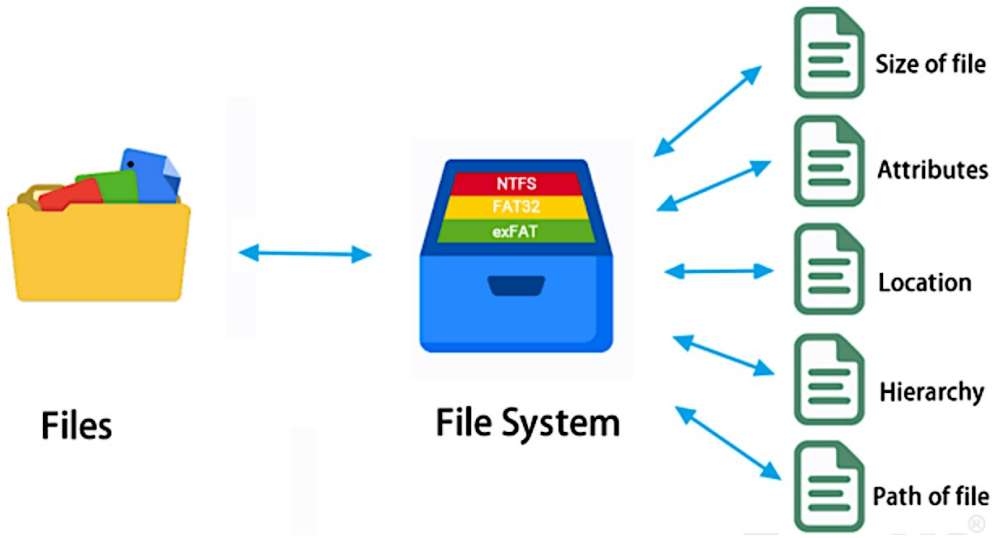
It provides an abstraction layer that allows users and applications to interact with stored data without needing to understand the underlying hardware details.

The primary objectives of a file system include:

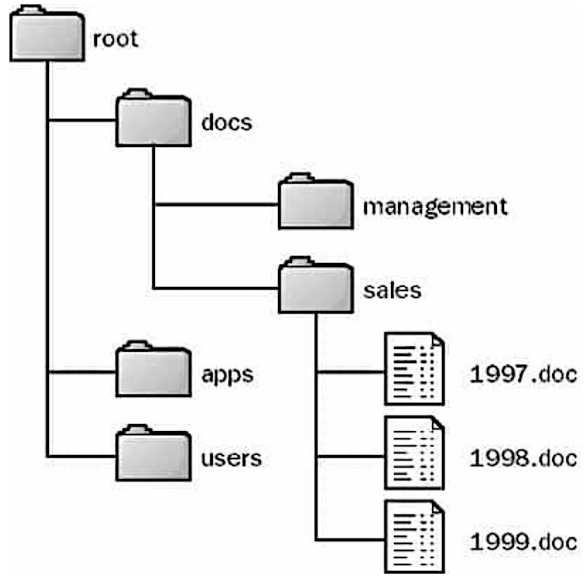
- **Storage Management:** Efficiently allocating and deallocating disk space to prevent fragmentation and maximize utilization.
- **Data Organization:** Structuring files in a hierarchical manner using directories and subdirectories for ease of access.
- **Access Control:** Enforcing security through permissions, ensuring only authorized users can read, write, or execute files.
- **Data Integrity:** Maintaining consistency through mechanisms like journaling to prevent corruption.
- **Performance Optimization:** Reducing access times through caching, indexing, and efficient block allocation.

Note:

Though they vary in design depending on the operating system they all share common principles in managing storage efficiently.



Source: https://www.easeus.com/partition-master/file-system.html?srsltid=AU7gw4Wd5hYuR2zp-ay4zVQ_4GLFUFsql8UqjVhuzYRhVv4VmBJHBADo



Source: https://networkencyclopedia.com/file-system/

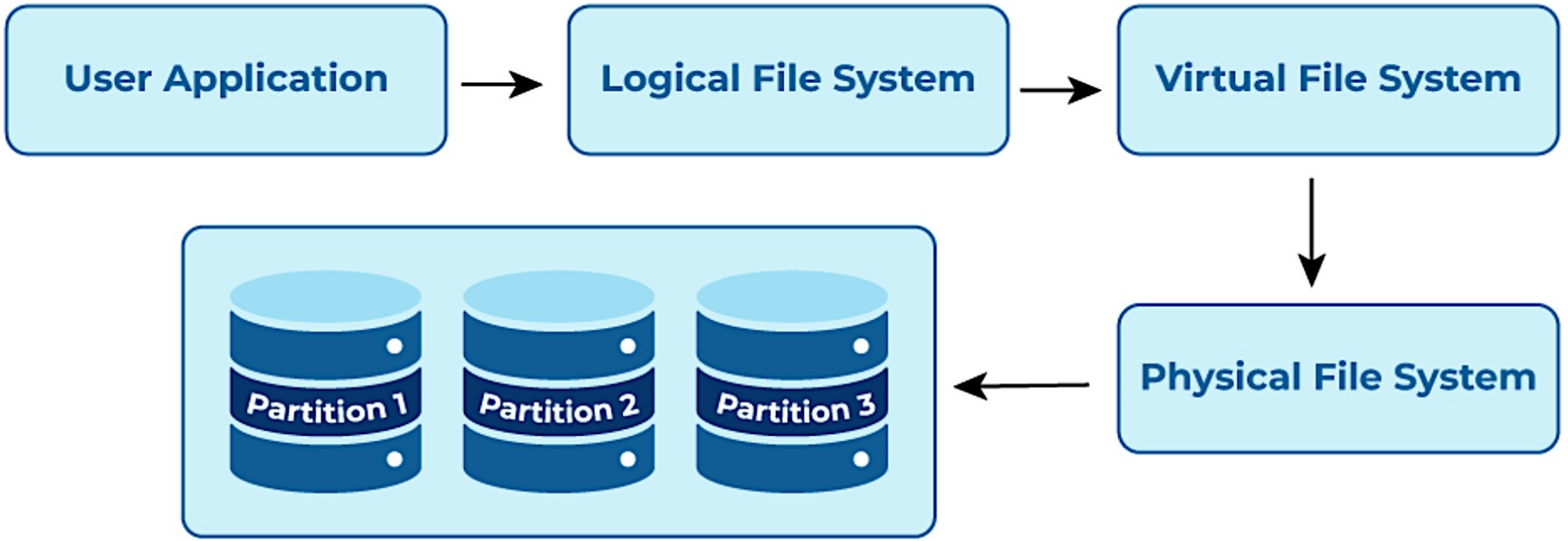
IV.3.1. File System Structure

A file system is typically structured in **layers**, each handling a specific aspect of file management, providing a structured way to store, organize and manage data on storage devices such as hard drives, SSDs and USB drives.

It acts as **a bridge** between the operating system and the physical storage hardware, allowing users and applications to perform CRUD Operations on files in an organized and efficient manner.

Layers used on file systems are:

- **User application:** The programs or software that request file operations like read, write, or delete (e.g. Microsoft word, VS Code, etc.)
- **Logical File System:** Manages metadata, file names, directories, and access permissions (e.g. ext4 for linux, NTFS for windows, APFS for macOs).
- **Virtual File System (VFS):** Acts as a bridge, allowing different file systems to work under a single interface.
- **Physical File System:** Handles the actual storage of data blocks on the disk.
- **Partition 1, Partition 2, Partition 3:** Divisions of the storage device where files are stored physically.



Source: https://www.geeksforgeeks.org/operating-systems/file-systems-in-operating-system/

Note:

- **Not all file systems** implement every layer explicitly, but **most modern operating systems use all these layers collectively to manage files.**
- Layers that are **universally present** are User application layer, Logical file system and Physical file system.
- Layers that depend on the operating system are:
  - Virtual File System (VFS):
    - Not part of every file system, but part of the OS kernel.
    - It's a middleware layer that lets the OS support multiple file systems (e.g., ext4, FAT32, NTFS) under one interface.
    - Found in Linux, macOS, and Android — but not necessarily in simpler embedded systems.
  - Partitions (Partition 1, 2, 3…):
    - These are optional and depend on how the storage is organized.
    - A file system can exist on a whole disk or within a partition.
    - Multi-boot systems or large drives often use partitions to separate OSes or data.

IV.3.2. File System Types

There are various of file systems depending on the platform used, among which we have:

- **Disk-Based file systems**
  - **FAT (File Allocation Table):**
    - Simple, used in older Windows systems.
    - Uses a file allocation table for block tracking.
  - **NTFS (New Technology File System):**
    - Supports journaling, encryption, and large files.
    - Uses a Master File Table (MFT) for metadata.
  - **ext4 (Fourth Extended File System):**
    - Default in Linux; supports journaling and extents.

- **HFS (Hierarchical File System):** A file system used by macOS.
- **APFS (Apple File System):** A new file system introduced by Apple for their Macs and iOS devices.

❑Network & Distributed File Systems

- **NFS (Network File System):** Allows remote file access over a network.
- **HDFS (Hadoop Distributed File System):** Designed for big data storage across clusters.

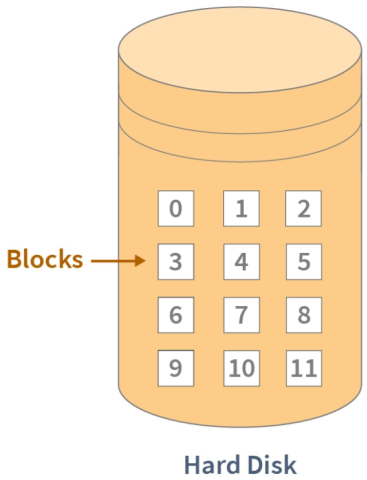
❑Special-Purpose File Systems

- **procfs (/proc):** Virtual file system providing process and system information.
- **tmpfs:** Stores files in RAM for fast access (volatile).

IV.3.3. File System Implementation

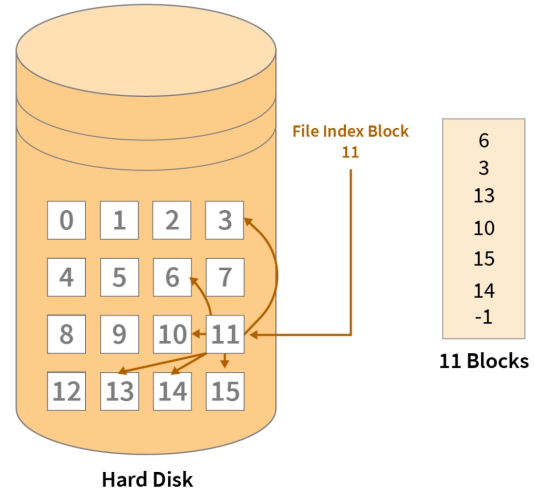
❑ **Allocation Methods**: Different strategies determine how files are stored on disk:

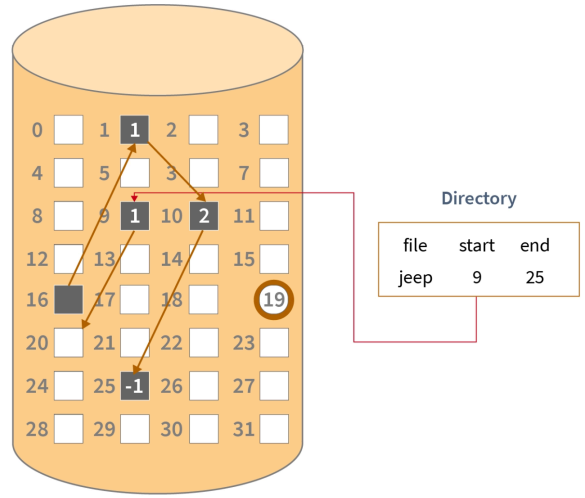
Method	Description	Advantages	Disadvantages
Contiguous	Files occupy consecutive disk blocks.	Fast sequential and random access (since block locations are predictable)	Suffers from external fragmentation, making it difficult to expand files.
Linked	Each block contains a pointer to the next block, forming a linked list	No external fragmentation; files can grow dynamically.	Poor random access performance (must traverse the entire chain).
Indexed	Uses an index block containing pointers to all data blocks of a file	Efficient random access; supports large files	Overhead of storing index blocks.



Directory

File Name	Start	Lenght	Allocated Blocks
abc,text	0	3	0,1,2
video.mp4	4	2	4,5
jtp.docx	9	3	9,10,11





❑ Free Space Management

It is the process File systems use to track free blocks for allocation.

Common methods include:

- ✓**Bit Vector (Bitmap):**
  - Each block is represented by a bit (0 = free, 1 = allocated).
  - Efficient for small to medium-sized storage.
- ✓**Linked list of free blocks:**
  - Free blocks are linked together in a list.
  - Simple but inefficient for large storage due to traversal overhead.
- ✓**Grouping and Counting:**
  - Stores ranges of free blocks to reduce metadata overhead.

IV.3.4. File System Protection

To ensure security of files, the following approach are often used for file protection:

- **Access control**
    - Read, Write, Execute permissions.
    - User-based access: Owner, Group, Others.
- **Security mechanisms**
    - Encryption
    - Access Control Lists (ACLs)
    - Role-Based Access Control (RBAC)

IV.3.5. File System Reliability & Performance

Most used techniques used are:

- **Journaling (Write-Ahead Logging)**
  - Records changes in a log before applying them to disk.
  - Prevents corruption in case of crashes (used in NTFS, ext3/ext4).
- **Caching and Buffering**
  - Buffer cache**:** Stores frequently accessed disk blocks in RAM.
  - Read-Ahead**:** Prefetches data for sequential reads.

- **RAID (Redundant Array of Independent Disks)**
  - ✓Improves performance and fault tolerance:
    - **RAID 0 (Striping):** Splits data across disks for speed (no redundancy).
    - **RAID 1 (Mirroring):** Duplicates data for fault tolerance.
    - **RAID 5 (Parity):** Distributes parity for error recovery.

Outline

- Generalities on files
- File systems
- **File Systems vs Directory Structure**
- Conclusion

IV.4. Directory Structure vs file systems

The Directory Structure **is part** of the File System, but the File System also handles lower-level tasks like block allocation, metadata management, and access control.

The Directory Structure is the **user-friendly interface**, and the File System as the **engine under this structure**.

Aspect	File System	Directory Structure
Definition	The entire framework for storing and managing files on a storage device	The hierarchical layout of folders and files
Scope	Includes metadata, block allocation, inodes, journaling, access control	Focuses on how files are organized and accessed by name
Function	Handles how data is stored, retrieved, and protected	Helps users and programs locate and manage files
Example Components	Inode table, superblock, data blocks, file allocation table	Root directory /, subdirectories like /home/christian/docs
User Interaction	Mostly behind the scenes	Directly visible and navigable by users

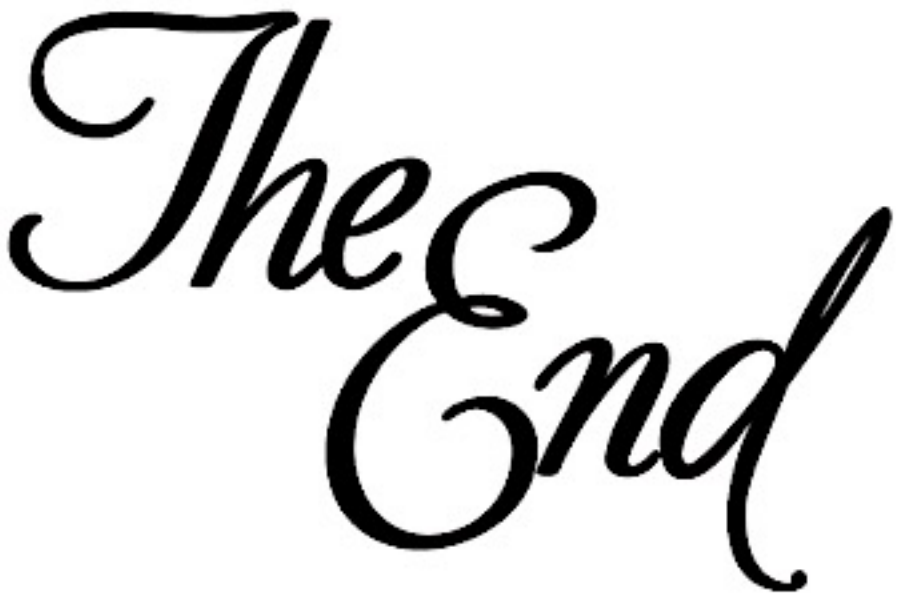
Outline

- Generalities on files
- File systems
- File Systems vs Directory Structure
- **Conclusion**

IV.5. Conclusion

File systems are critical for organizing, storing, and retrieving data. Different access methods and directory structures serve different needs.

While the Implementation choices affect performance, reliability, and scalability, it appear crucial to ensure a protection and recovery mechanisms to maintain data integrity and security.


The End